Instituto Federal do Maranhão
Campus Grajaú

Tecnologia em Análise e Desenvolvimento de Sistemas

Lista de Exercícios

Aula 03 — Encapsulamento, Modificadores de Acesso, `Getters`, `Setters` e Validação de Dados

Disciplina	Programação Orientada a Objetos
Professor	Welington Galvão Rodrigues
Turma	ADS 2026.2
Data	22/08/2026 (Sábado — reposição de 19/08)
Horário	14h40 – 18h15
Valor total	10,0 pontos
Aluno(a)	_____
Matrícula	_____

Instruções Gerais

- Responda todas as questões com atenção e clareza.
- Nos exercícios de código, os atributos devem ser `private`, com *getters* e *setters* `public` conforme solicitado.
- Sempre que houver validação, indique o comportamento esperado quando um valor inválido for informado.
- É permitido o uso de anotações e material de aula.
- Esta lista compõe a nota de atividades da 1ª Etapa.
- Entregar ao final da aula ou conforme orientação do professor.

Bloco	Conteúdo	Questões	Valor
A	Conceitos	01 – 02	2,0 pts
B	Questões de Código	03 – 10	8,0 pts
Total			**10,0 pts**

(2 questões × 1,00 pt = 2,0 pts)

Questão 01 (1,00 pt)

O que é encapsulamento na Programação Orientada a Objetos? Explique, com um exemplo **diferente** dos vistos em aula, por que deixar atributos públicos pode comprometer a integridade dos dados de um objeto.

Questão 02 (1,00 pt)

Compare os quatro modificadores de acesso do Java (`private`, padrão/*package-private*, `protected` e `public`), indicando de onde cada um pode ser acessado (mesma classe, mesmo pacote, subclasse, qualquer lugar).

(8 questões × 1,00 pt = 8,0 pts)

Questão 03 (1,00 pt)

Encapsule uma classe `Produto`, com os atributos `nome` (`String`) e `preco` (`double`). Implemente *getters* para ambos e um *setter* para `preco` que **rejeite valores negativos**, mantendo o valor anterior nesse caso.

Questão 04 (1,00 pt)

Encapsule uma classe `Aluno`, com os atributos `nome` (`String`) e `nota` (`double`). O *setter* de `nota` só deve aceitar valores entre `0` e `10` (inclusive); fora desse intervalo, rejeite e mantenha o valor anterior.

Questão 05 (1,00 pt)

A classe abaixo tem atributos públicos, sem nenhum controle de acesso:

```java
public class Retangulo {
    public double largura;
    public double altura;
}
```

Refatore-a para uma versão encapsulada: atributos `private`, *getters* para ambos e *setters* que **rejeitem valores menores ou iguais a zero**.

Questão 06 (1,00 pt)

Encapsule uma classe `Termometro`, com o atributo `temperatura` (`double`). O *setter* só deve aceitar valores entre `-50` e `60` graus; fora dessa faixa, exiba uma mensagem informando que o valor é inválido e mantenha a temperatura anterior.

Questão 07 (1,00 pt)

Encapsule uma classe `Estoque`, com o atributo `quantidade` (`int`, privado, inicia em `0`). Implemente os métodos `adicionarItens(int qtd)` e `removerItens(int qtd)`. O método `removerItens` não pode deixar `quantidade` ficar negativa — se a remoção solicitada for maior que a quantidade disponível, rejeite a operação e exiba uma mensagem.

Questão 08 (1,00 pt)

Encapsule uma classe `Email`, com o atributo `endereco` (`String`, privado). O *setter* de `endereco` só deve aceitar valores que **contenham o caractere `@`**; caso contrário, rejeite e mantenha o valor anterior (use o método `contains` de `String`).

Questão 09 (1,00 pt)

A classe abaixo compila, mas tem um problema de design:

```java
public class ContaBancaria {
    private double saldo;

    public double getSaldo() {
        return saldo;
    }

    public void setSaldo(double saldo) {
        this.saldo = saldo;
    }
}
```

(a) Qual é o problema, mesmo com `saldo` sendo `private` e havendo um *getter* e um *setter*?

(b) Reescreva o *setter* `setSaldo`, adicionando validação para que o saldo nunca possa se tornar negativo através dele.

Questão 10 (1,00 pt)

A classe abaixo representa um funcionário, mas ainda não está encapsulada:

```java
public class Funcionario {
    public String nome;
    public double salario;
}
```

Refatore-a para uma versão encapsulada:

- Atributos `private`, com *getters* para ambos;
- *Setter* de `salario` que rejeite valores menores ou iguais a zero;
- Um método `aumentarSalario(double percentual)`, que só aplica o aumento se `percentual` for maior que zero (caso contrário, rejeite a operação e exiba uma mensagem).

"Encapsular é proteger — esconda os dados, exponha o comportamento."

Boa sorte! — Prof. Welington Galvão Rodrigues